How Our Waste Assistant Works: Model and System Guide

Nov 7, 2026 · Anar, Naura, Shukurullo, Vitou (equal contribution)

The model part works end to end: a person asks about waste in their own language, and the system answers with a bin or a collection day, plus its sources. On our 22 hand-tested questions it gave 17 correct answers, up from 10 for the first version, and showed no wrong bin.

Those 22 questions are developer tests, not the official evaluation. They were also used to adjust the system, so the official numbers will come from the dev and test sets.

This guide explains the system for readers who are new to RAG. You can read it from the top, or jump to What each area needs next. The Word list at the end explains every technical term.

Area	Status today
Answering "which bin?"	Works with sources, tested mostly in English. Chinese and Russian each got the right bin on their one test, the one Uzbek test is refused, and Vietnamese is untested
Answering "when and where?"	Works from the table of 10,187 district rows
Refusing when unsure	Works: it can ask back, refuse, or give the office phone number
Repository clean-up	Done: 43 automated tests pass, and the 147 stored runs gave the same results after the clean-up
Website connection	Not built yet: the `rag/service.py` layer for the website is the next stage
Tuning on real test cases	Waiting for the dev set

RAG in plain words

RAG means the system looks up the official rules first, then answers from what it found instead of from memory.

Think of a closed-book exam and an open-book exam. A plain chatbot sits a closed-book exam: it answers from memory and can sound sure while being wrong. RAG makes it an open-book exam: we find the right pages first and tell it to answer only from those pages.

Letter	Stands for	What it means in our project
R	Retrieval	Find the pieces of the official documents that match the question
A	Augmented	Add those pieces to the instructions we send to the language model
G	Generation	The language model writes the answer from those pieces and cites them

Our project needs this for three reasons:

- **The rules are local.** They come from Korean government guides and a district table, and a general chatbot does not reliably know them.
- **The sources are in Korean, but users are not.** Someone can ask in English and the answer must still come from a Korean rule.
- **Mistakes cost the user.** A wrong bin can mean uncollected waste or a fine, so every answer needs a source they can check.

Three ideas you will see everywhere

Chunk. A chunk is one small piece of a document, and search returns chunks rather than whole documents. We have 403 of them from the 2018 national guide and the 2026 rules. This is the real chunk for pizza boxes:

```
[분리배출 품목] 피자박스
구분: 종이 (paper)
배출방법: 기름에 오염된 내부 종이는 종량제봉투로 배출
출처: 환경부 재활용품 분리배출 가이드라인 (2018) p.33
```

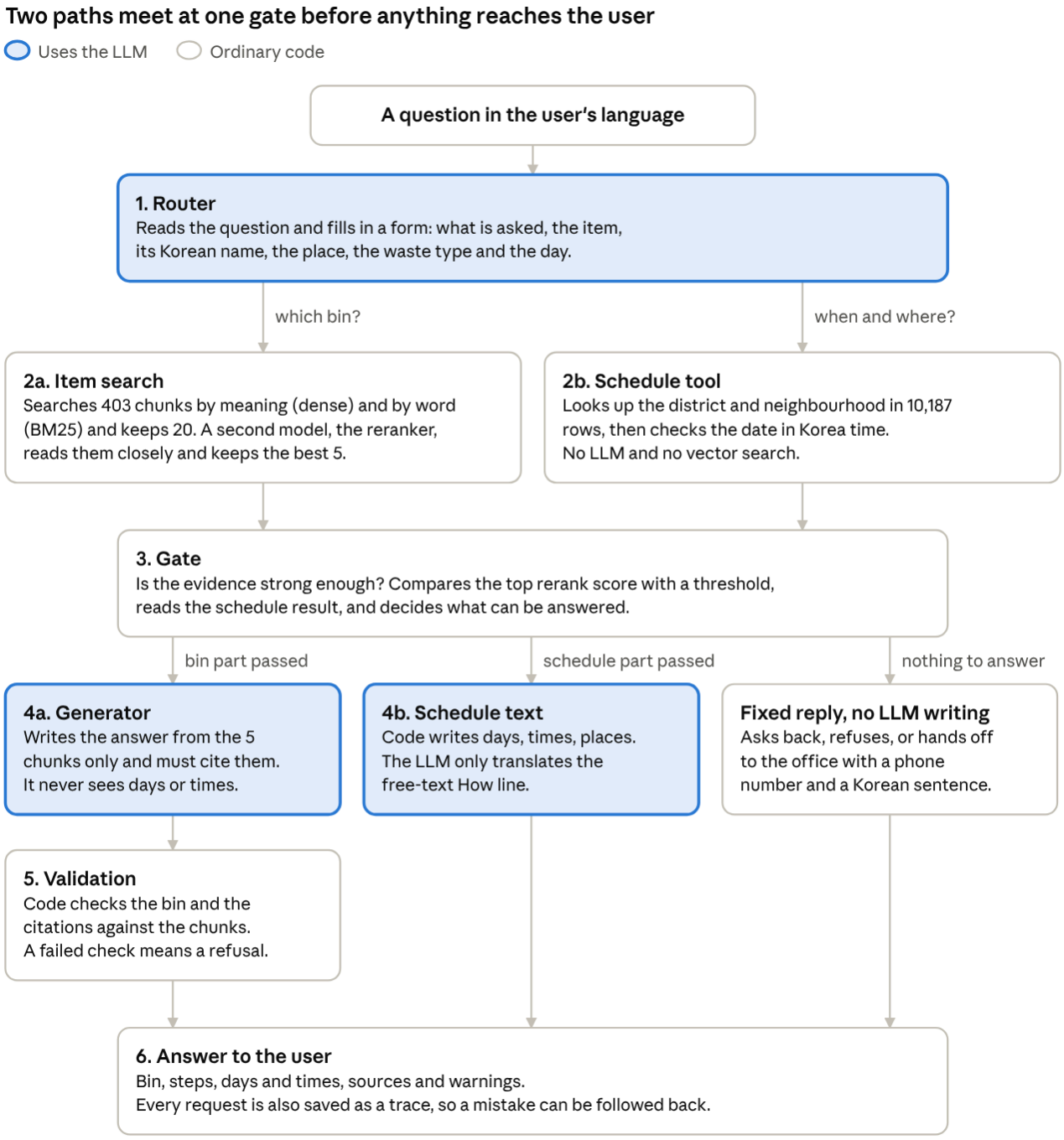
In English: the item is a pizza box, its category is paper, and inner paper stained with oil goes in the pay-as-you-throw bag. The source is the 2018 guide, page 33.

Embedding. An embedding turns text into a list of numbers that represents its meaning. Ours has 1,024 numbers per chunk. Texts with similar meanings get similar numbers, even in different languages, so "pizza box" can find 피자박스.

LLM. An LLM (large language model) is the part that reads and writes text, like the model behind a chatbot. We run `qwen2.5:7b` on our own laptop through Ollama, so questions stay on the machine and there is no API cost.

The whole system at a glance

A question takes one or both of two paths, and both meet at the same gate. Code sits between the LLM and the user at the router, the gate and validation, so a bin from the LLM is never shown unchecked.



How one question moves through the system

When the gate finds nothing it can answer, the system skips the generator and replies with fixed text. The next section follows a real question through each numbered stage.

Stage by stage: one real question

This section follows one question through the system, using real output from a run on 2026-11-07: **"Where do I throw away a pizza box?"**

1. Router: work out what is being asked

The first LLM call turns the question into a filled-in form. The documents are in Korean, so the router also writes a Korean search query. The code is in `rag/router.py`.

Field	Value for this question	What it is for
`intent`	`item`	Picks the path: `item`, `schedule`, `both` or `out_of_scope`
`item`	pizza box	The thing being thrown away
`item_ko`	피자 상자	The same item in Korean
`ko_query`	피자 상자 분리배출 방법 ("pizza box disposal method")	The Korean search text
`district`, `dong`	empty	The place, if the user gave one
`waste_type`	`recycling`	General, food, recycling or bulky
`day_ref`	empty	"Today", "tonight", a weekday, or nothing

Code then checks the form, because a rewrite can quietly change the question. A place that is not in the question is dropped, so the system never guesses a location. If the output is invalid twice, it searches with the original question only.

2. Item search: find the best 5 of 403 chunks

Search works like a funnel with two steps. The code is in `rag/retrieve.py`.

1. **Wide search.** Dense search finds chunks close in meaning, using embeddings. BM25 search finds chunks that share the same words. RRF then merges the lists, and the top 20 go forward.
2. **Careful re-read.** A second model, the reranker, reads the question and each chunk together and scores the match from 0 to 1. The top 5 go forward.

The system searches with both the original question and the Korean rewrite. That way one bad translation cannot sink the search: when the router turned "mattress" into 고기저귀 ("meat diaper"), the original question still found the right chunk.

These are the 5 chunks chosen for our question:

Final rank	Chunk (in English)	Rank before rerank	Rerank score
1	Pizza box, 2018 guide p.33	1	0.990
2	Corrugated cardboard, 2026 rules	2	0.586
3	Paper: books, newspapers, boxes, 2026 rules	3	0.338
4	Mayonnaise container, 2018 guide p.29	12	0.164
5	Newspapers and other paper, 2018 guide p.15	7	0.116

The right chunk is far ahead of the rest. Rank 4 is not relevant, which is why the top 5 are treated as candidates and the model must say which ones it used.

3. Gate: is there enough evidence?

Before the LLM writes anything, code compares the top rerank score with a threshold called tau (τ), set to 0.15. Pizza box scores 0.990, so it continues. A score below 0.15 gives a fixed refusal with no LLM call. The code is in `rag/gate.py`, and Safety covers this in more detail.

4. Generator: write the answer from the chunks

The second LLM call receives its rules, a few worked examples and the 5 chunks. It must return a fixed JSON form. Temperature is 0 and the seed is fixed, so repeat runs stay as close to identical as possible. The code is in `rag/generate.py`.

The LLM can only read 8,192 tokens at once. We keep 768 for its answer, which leaves 7,424 for the input, and this question used 2,750. If the evidence ever does not fit, the lowest-ranked chunks are dropped and the drop is recorded.

The real output was:

```
{
  "status": "ANSWER",
  "bin": "종이",
  "bin_en": "paper",
  "how_to_prepare": ["remove any food residue", "fold it flat"],
  "answer_text": "Fold the pizza box flat and remove any food residue. Then,
put it in the paper bin (종이) [1].",
  "citations": [1],
  "missing": ""
}
```

The `[1]` points to chunk 1 in the table above, so the user can trace the answer to the 2018 guide, page 33.

5. Validation: check before showing

Ordinary code, not the LLM, checks the answer. The code is in `rag/validate.py`.

- The bin must be one of our 18 allowed bins.
- The cited chunks must state that bin.
- The bin field must match the bin named in the text.
- The answer must cite at least one chunk.

If a check fails, the answer becomes a refusal. Pizza box passes, because chunk 1 says 종이 (paper).

This run also shows a limit. Validation checks the bin, not the steps. The chunk says oil-stained inner paper goes in the general-waste bag, but this answer left that out and validation cannot notice. See Known limits.

6. Final answer and trace

`rag/answer.py` adds the English bin name, the source and any warnings. Here the warning is: "Based on the 2018 national guideline; your district may have newer local rules."

Every request is also saved as one JSON record in `traces/<date>.jsonl`. It holds the router form, all 20 candidates with their scores, the raw LLM output, every check and the timings. When an answer is wrong, the trace shows the earliest stage that failed.

A typical bin answer takes about 19 seconds (the median on our tests). Most of that is LLM time: 13.6 seconds for the generator and 6.8 for the router.

Two kinds of question

Bin questions use search, because the rules are written as text. Schedule questions use an exact table lookup, because the answer must be right for one specific place.

	"Which bin?"	"When and where?"
Source	403 chunks from the national guides	10,187 rows covering 222 districts in 16 provinces
How it is found	Dense + BM25 search, then rerank	Exact lookup by district and neighbourhood
Who writes the answer	The LLM, from the chunks	Code; the LLM only translates the free-text "How" line
Time per answer	About 19 seconds	3 to 10 seconds
Code	`rag/retrieve.py`, `rag/generate.py`	`rag/schedule_tool.py`, `rag/schedule_text.py`

A question can need both, such as "Where do PET bottles go, and when is recycling collected in my area?" The router marks it `both`, each path runs, and the two parts are joined into one answer.

A real schedule answer

We asked: **"I live in Seongdong-gu Haengdang-dong. Can I put food waste out tonight?"** The run was on Saturday 2026-11-07, and the answer began:

```
Food waste (음식물쓰레기): Tonight (Sat 2026-11-07): no
Food waste (음식물쓰레기): Mon, Wed, Fri, 20:00-24:00 [1]
Where: Door-to-door collection (in front of your house) (문전수거) - 집앞
```

The tool checked the real calendar date in Korea time, so "tonight" was answered correctly. A "How" line followed, machine-translated from Korean, and it reads roughly. The answer also warned that this district row is dated 2020-12-08.

Why not use search for schedules too?

Experiment E4 tested this on 27 schedule questions. The rows look almost the same to a search model, so it mixes up places.

- Plain vector search showed the wrong place on 18 of the 27 questions.

- A resident of 강동구, which has no data, was given days from 울산 북구, 부산 강서구 and 서울 강북구.
- The exact lookup showed the wrong place on 0 of the 27.

Why does code write the schedule text?

Even with the right row, the LLM made mistakes when it wrote the answer. It dropped days and zones, and it called food waste "general waste bag". Only 12 of 19 answers were fully correct.

Now the LLM never sees days, times or places, so it cannot change them. Code writes those parts, and 19 of 20 answers were correct on the same kind of test. The one partial was a translated "How" line that lost the meaning of a pay-as-you-throw bag.

What happens when a place is unclear

The answer shows at most 4 rows, so some places need a follow-up:

Situation	What the system does	Example
The name exists in several provinces	Asks which one	중구 exists in 5 provinces
A district has more than 4 zones and no neighbourhood was given	Asks for the neighbourhood	Lists up to 30 options
A name covers several smaller areas with different days	Asks which one	오빈리: 오빈1리, 오빈2리, 오빈3리
One name has many collection points with different days	Refuses and gives the office phone	고흥군 도덕면: 287 points, Monday to Friday
The rows differ in place but share the same days	Answers	정선읍
The district has no data	Refuses and points to the office	강동구, 도봉구

Safety

A wrong bin costs the user more than "I don't know", so the system refuses when it is unsure. Every answer ends in one of four statuses. The examples below come from real runs on 2026-11-07.

Status	Meaning	Real question	What the user gets
`ANSWER`	Evidence was found and the checks passed	"Where do I throw away a pizza box?"	Bin, steps and sources
`CLARIFY`	One detail is missing	"When is recycling collected?" (no place given)	A question back, sometimes with options
`INSUFFICIENT_CONTEXT`	No answer it can trust	"How do I throw away nail polish bottles?"	"I will not guess", plus a handoff
`OUT_OF_SCOPE`	Not about household waste in Korea	"How do I scrap my car?"	A fixed message saying what it can help with

`INSUFFICIENT_CONTEXT` is the system's word for abstaining, which means choosing not to answer.

Several layers of protection

No single check catches everything, so each layer stops a different mistake. Most of them are ordinary code, which cannot be talked out of a rule the way an LLM can.

Layer	Runs as	What it stops	Real case
Router checks	Code	Guessed places and rewrites that lost the item	A district not named in the question is dropped
Gate	Code	Answers built on weak evidence	Dead hamster: top score 0.001, refused
Generator rule	LLM	Answers the chunks do not support	Nail polish: the model said it found no rule
Validation	Code	A bin the evidence does not state	Styrofoam: the model said 종량제 봉투, the evidence said 발포합성수지, refused
Schedule rules	Code	Stating days from a small part of the rows	고흥군 도덕면: office phone instead of an answer

Layer	Runs as	What it stops	Real case
Age warnings	Code	Old data shown as if it were current	6,236 rows (61%) are over a year old, and the answer says so

The nail polish case shows why the layers matter. Its top score was 0.331, which passed the gate even though no rule exists, because the search found other bottles. Only the generator stopped it, and we should not rely on that every time.

Handoff: what happens after a refusal

When the system cannot answer, it does not leave the user stuck. The handoff gives them three things, built by code in `rag/handoff.py`:

- **Who to contact:** the district office and phone number, taken only from the district data and never invented.
- **What to say:** a short Korean sentence they can show or read at the office. If the Korean item name looks broken, the sentence uses the user's own words for the item.
- **What was already checked,** so they do not have to start again.

What we tested and learned

Eight comparisons shaped the design, and most of them confirmed the plan. The clearest result is that an English-only embedding cannot search Korean rules. The biggest surprise is that hybrid search did worse than dense search alone.

All of these use developer probes: 22 general questions and 27 schedule questions. 13 of the 22 have a known right chunk, and the search scores use those 13. The numbers show direction, not proof.

Question	Options compared	Result	Decision
Does the full system beat the first version?	First version (dense search only, no router, reranker or gate) vs full system	Correct / partial / wrong: 10 / 3 / 9 vs 17 / 3 / 2. Median time: 10.7 s vs 19.0 s	Keep the full system. Its 2 wrong results are refusals, not wrong bins
Which LLM size?	`qwen2.5:3b` vs `qwen2.5:7b` (12 probes)	7 of 12 correct vs 9 of 12	Use 7b

Question	Options compared	Result	Decision
Which embedding? (E2)	`bge-small-en` vs `bge-m3`	Recall@5: 0.000 vs 1.000	Use `bge-m3`
Which search method? (E3)	Dense, BM25, hybrid	Recall@5: 1.000, 0.462 (0.615 with character bigrams), 0.923	Hybrid lost to dense here. Still open, see below
Does reranking help? (E5)	Hybrid with and without the reranker	MRR: 0.703 vs 0.846. Recall@5: 0.923 vs 1.000	Keep it: it repairs hybrid and gives the gate a usable score
How to find schedules? (E4)	Exact lookup, vector search, two filtered searches	Expected behaviour: 27 of 27, 20 of 27, 22 of 27 for each filter	Keep the exact lookup
Which prompt?	`answer_v2` vs `answer_v3`	Keyword score: 20 of 22 vs 18 of 22. v3 removed invented steps but gave a wrong bin for batteries	Keep v2 for now. E1 on dev should re-test this
A Korean LLM for the router?	`qwen2.5:7b` vs EXAONE 3.5	MRR: 0.846 vs 0.723. EXAONE turned chicken bones into "glass bottle"	Not adopted. This ran on old data, so a re-run would be fairer

Two scores appear in this table:

- **Recall@5:** how often the right chunk is in the top 5. A score of 1.000 means always.
- **MRR:** how high the right chunk ranks. A score of 1.000 means always first, and 0.5 means about second place.

Why hybrid is still the default. It is not the best row, so this needs explaining. Most of our probes are simple and not in Korean, which suits dense search. BM25 should help with exact Korean terms, and the probes barely test those. If the dev set shows no gain, we drop BM25.

E1, E6, E7 and E8 have not run yet. They wait for the dev set or for planned work.

Three lessons from this work

Fix the data before tuning. Food-waste rules sit in tables on pages 17 and 18 of the guide, and PDF extraction scrambled them. No model change could fix that, so the tables were typed in by hand as 9 chunks. The keyword score on the same probes then rose from 18 of 22 to 21 of 22.

Read the answers, not only the scores. The keyword checker gave the first version 16 of 22, while reading the answers found only 10 correct. An automatic score needs a human audit.

Good retrieval does not mean a good answer. In E4 the right row was found for every answerable question, yet the first answers still had wrong or missing details. A metric that stops at retrieval could not have shown this.

Where things live

The repository was reorganised on 2026-11-07, so that code folders hold only code and data lives under `data/`. Paths in older notes may use the old names.

Folder	What it holds
`data/raw/`	Official sources, never edited: the district CSV and government PDFs
`data/manual/`	Written by hand: the glossary and the food-waste table
`data/interim/`	Output of the PDF parser
`data/processed/`	Clean files the system reads, built by `preprocess/`
`data/reports/`	Data-quality and search-index reports
`preprocess/`	Turns raw data into processed data
`ingest/`	Builds and verifies the search index
`rag/`	The system itself
`rag/prompts/`	Versioned instructions for the LLM
`eval/`	Evaluation code, with cases and metrics started
`experiments/`	Experiment settings, probe questions and raw results
`tests/`	43 fast tests that need no models or Ollama

Folder	What it holds
`docs/`	Written results: `PHASE1_NOTES.md` and `PHASE2_NOTES.md`
`notebooks/`	Data exploration notebooks
`qdrant_data/`, `traces/`	Generated on each machine and not stored in git

Old name	New name
`Data/`	`data/raw/`
`Vitou_EDA/`	`notebooks/schedule_eda/`
`item_guide/build_item_chunks.py`	`preprocess/parse_item_guides.py`
`python -m rag.pipeline`	`python -m rag`

If you only read three files in `rag/`, read these:

- `rag/pipeline.py` runs all the stages in order.
- `rag/config.py` holds every setting, so an experiment changes one value in one place.
- `rag/paths.py` holds every file location.

One rule keeps the structure clean: `preprocess`, `ingest`, `eval` and `experiments` may import `rag`, but `rag` imports none of them. A test enforces this.

Try it yourself

You need Python 3.11 or 3.12, Ollama and about 10 GB of free disk space for the models. We developed on an Apple M3 Pro with 36 GB of memory. The commands below run from the repository root.

```
# 1. Python environment
python3.12 -m venv .venv
source .venv/bin/activate
pip install -r requirements-dev.txt

# 2. Local LLM (keep Ollama running)
ollama pull qwen2.5:7b

# 3. Build and check the search index. The first run downloads about 2.3 GB.
python -m ingest.build_index
python -m ingest.verify_index

# 4. Ask a question. The first question downloads the reranker, about 2.3 GB.
python -m rag "Where do I throw away a pizza box?" --lang en
python -m rag "When can I put out food waste?" --district 성동구 --dong 행당동

# Add --show-trace to see every stage: router form, scores, checks, timings
python -m rag "Where do I throw away a pizza box?" --lang en --show-trace
```

These commands do not need the LLM:

```
# 43 tests, about 1 second
python -m pytest

# Schedule lookup only
python -m rag.schedule_tool 성동구 --dong 행당동 --waste-type food

# Search only
python -m rag.retrieve "택배 스티로폼 상자" --retrieval hybrid --rerank
```

What each area needs next

The remaining work runs in this order: website-ready layer, evaluation cases, tuning, final test, report and slides, then the website. The full task list is in `PLAN.md` section 10.4.

Data

The system reads the clean files in `data/processed/`. They are built from the raw sources by `preprocess/`, which was added so the model would not be measured on bad data. The earlier fix in `notebooks/schedule_eda/` also helped: comparing against it exposed area lists like `상계6.7동` that were being missed.

Please review these before evaluation starts, because the right answers in the test cases depend on them:

- [ ] The 9 hand-typed chunks in `data/manual/food_waste_table_2018.json`, against pages 17 and 18 of the guide
- [ ] The 36 terms in `data/manual/glossary.json`
- [ ] The data fixes listed in `docs/PHASE2_NOTES.md` section 1

The data branch still needs merging. Its `.gitignore` uses old paths such as `/Data/image/`, so those will need updating.

Evaluation

One call runs a test case and returns the full trace, including every candidate, score and check:

```
from rag.pipeline import Pipeline

pipeline = Pipeline()
case = {"case_id": "c017", "lang": "en", "location": None,
        "question": "Where do I throw chicken bones?"}
trace = pipeline.run_case(case)
```

Cases go in `eval/cases/dev.jsonl` and `eval/cases/test.jsonl`, one JSON object per line. `PLAN.md` sections 7.1 and 7.2 give the format and the target of about 100 cases. The example below is wrapped to fit the page.

```
{"case_id": "c017", "version": "v1", "split": "dev", "slice": "item_tricky",
 "lang": "en", "question": "Where do I throw chicken bones?",
 "location": null, "expected_behavior": "answer",
 "expected_bin": ["종량제봉투"],
 "expected_chunk_ids": ["<uuid of the matching chunk>"],
 "expected_schedule": null,
 "reference_facts": [
   "Chicken bones go in the general-waste bag, not food waste."],
 "notes": "Common mistake: putting bones in food waste."}
```

`experiments/run_experiment.py` already runs one experiment over a case file, and it refuses a test file unless you pass `--allow-test`. `eval/metrics.py` has first-look summaries only. `eval/make_cases.py` and `eval/run_eval.py` still need building.

Three decisions are needed from you or the team:

- [ ] Who writes the test cases. It should be someone who does not tune the system, so the test stays blind.
- [ ] Whether to sample schedule cases by row or by district. 정선군 and 고흥군 alone are about a third of the table.

- [ ] The dev and test split, which must be frozen before any tuning starts.

Website

This call returns everything the page needs:

```
from rag.pipeline import Pipeline

pipeline = Pipeline()   # loads two models, so create it once and keep it
answer = pipeline.answer("Where do PET bottles go?", lang="en",
                         district="성동구", dong="행당동")
```

Field	How to use it on the page
`status`	Choose the layout: `ANSWER`, `CLARIFY`, `INSUFFICIENT_CONTEXT` or `OUT_OF_SCOPE`
`answer_text`	The main text. It can have several lines, so keep the line breaks. `[n]` points to a citation
`bin`, `bin_en`	The bin name in Korean and English
`how_to_prepare`	The preparation steps as a list
`citations`	The sources: number, kind, reference and source name
`schedule`	Days, times, places and zones, with next dates, office and phone
`clarify`	A question plus up to 30 options, which suit buttons
`handoff`	Office, phone, the Korean sentence and what was checked
`notes`	Warnings about source year and data age, which should stay visible
`missing`	What could not be answered. Today this is the only place that says one part of a mixed question failed
`trace_id`	The ID of this answer, needed for saving feedback
data_date, office_phone	The date of the data shown and the office phone number

Three things to plan for:

- **Speed:** answers took 3 to 28 seconds in the runs for this guide, so the page needs a clear loading state.
- **Setup:** the page needs Ollama running on the same machine.
- **Language:** `clarify.question`, `handoff.contact` and `notes` are in English only for now.

A simpler `rag/service.py` is planned as the one module the website imports. It is not built yet, and the same goes for feedback saving and the offline mode. `docs/answer_examples.json` will then replace the mock answers with one real answer per status.

The website branch is not merged yet. The mock page will need `OUT_OF_SCOPE`, clarify options, handoff, source cards, all six languages and the full location list.

Everyone

These tasks need real people, and the report must not imply they were generated.

Task	Who	Needed by
6 to 10 real questions from international students	Team	Evaluation cases
Native-speaker check of the glossary, labels and Chinese text	Speakers of zh, vi, uz, ru	Before the report claims those languages work
Check of paraphrased test cases	Team	Evaluation cases
Audit of about 20 answers by two independent raters	Two members	Final evaluation
Deadline, presentation date and demo format	Team	Now: this sets how much optional work fits

Known limits

The system refuses many of its own mistakes, but some still get through and some good answers are refused. These are the open problems, listed by the stage that fails first. All of them need testing on the dev set.

Problem	Stage	Real example	Next step
Uzbek questions are refused even when search works	Gate	Chicken bones in Uzbek: right chunk ranked first, but the score was only 0.006	Measure by language on dev, then try a glossary lookup or a stronger translator
A fluent but wrong rewrite passes every check	Router	"Mattress" became 고기저귀 ("meat diaper")	Compare the rewrite with the question, or check the item appears in the evidence
Mixed-language text counts as Korean	Router	Nail polish became 네일 pol 리스 병. Search can still use such text, but the handoff sentence now shows the user's own words instead	Make the check used for search stricter, and measure that on dev
Preparation steps are not checked	Validation	The model told users to "puncture the cardboard" of a pizza box, and today's run left out the oily-paper rule	Check steps against the cited chunks, and compare prompts in E1
A wrong citation can pass if the bin matches	Validation	Prompt v3 cited a metal-can rule for batteries	Check that the cited chunk is about the asked item
Tau rests on very little data	Gate	It was set from 15 probes with only 2 negatives, and nail polish passed it at 0.331	Choose it on dev in E5
Hybrid search is not better than dense	Search	Recall@5: 0.923 vs 1.000	Run E3 and E5 on dev with exact Korean terms, and drop BM25 if there is no gain
Translations of the "How" line are weak	Translation	Vietnamese, Russian and Uzbek readers get this line in English	Native-speaker review of the 50 most common lines, which cover about 89% of fields

Problem	Stage	Real example	Next step
Some fixed text is English only	Output	`clarify.question`, `handoff.contact` and `notes`	Write them in code for all six languages
Answers are slow	System	About 19 seconds at the median	Measure each stage, and try a smaller router model
The source data has gaps	Data	강동구 and 도봉구 have no schedule, and 61% of rows are over a year old	The system warns users, but only new official data can fix this

The biggest limit is evidence. Every number in this guide comes from a small set of hand-picked questions, so the official results may be lower.

Word list

Term	Plain meaning
Abstain	Choose not to answer. Shown as `INSUFFICIENT_CONTEXT`
Baseline	A simple first version kept for comparison. Ours is the Phase 1 system
Bigram	A pair of neighbouring characters, used as a search word
BM25	Keyword search that scores chunks by the words they share with the question
Candidate	A chunk the search found but has not yet chosen
Chunk	One small piece of a document, and the unit that search returns
Citation	A marker like `[1]` that links a statement to its source
Corpus	The full set of chunks. Ours has 403
Cross-encoder	A model that reads the question and a chunk together. Our reranker is one

Term	Plain meaning
Dense search	Search by meaning, using embeddings. Also called vector search
Dev set	Test cases used while tuning the system
Embedding	A list of numbers that represents the meaning of a text
Evidence	The chunks or rows sent to the LLM for one question
Few-shot	Giving the LLM a few worked examples inside its instructions
Gate	The check that decides if there is enough evidence to answer
Generator	The LLM step that writes the answer
Gold chunk	The chunk we know contains the right answer for a test case
Handoff	Passing the user to a person, with a phone number and a Korean sentence
Hybrid search	Dense search and BM25 combined
Index	The stored embeddings, ready to search
JSON schema	A fixed form that the LLM's output must follow
Kiwi	A Korean language tool that splits words into smaller units for BM25
LLM	Large language model, the part that reads and writes text
MRR	Mean Reciprocal Rank: how high the right chunk ranks, where 1.000 means always first
Ollama	The program that runs the LLM on our own machine
Probe	A hand-written developer test question, not part of the official evaluation
Prompt	The instructions and text sent to the LLM
Qdrant	The database that stores our embeddings

Term	Plain meaning
Recall@5	How often the right chunk is in the top 5
Reranker	A slower, more careful model that re-scores the search results
Rewrite	The Korean search query written by the router
Router	The first LLM step, which works out what the question is asking
RRF	Reciprocal Rank Fusion: merges ranked lists using each chunk's position in them
Seed	A number that makes the LLM's random choices repeatable
Slice	A group of test cases of one type, such as tricky items
Stale	Data older than one year
Tau (τ)	The minimum rerank score needed to answer, currently 0.15
Temperature	How much randomness the LLM uses. 0 is the most predictable
Test set	Test cases kept private until the one final run
Token	A small piece of text, often a word or part of one, that LLMs count
Trace	The saved record of everything that happened for one request
Validation	Code checks on the LLM's answer before the user sees it